

Robotic Systems in Manipulation - Phase I: Kinematics

Rodrigo Sales Freire dos Santos (ist1119658)

Mikhail Mohamed Shueb (ist1104014)

1 Step 1: Kinematic Model (D-H Convention)

In this step, the kinematic model of the KUKA LBR MED robot was developed using the Denavit-Hartenberg (D-H) convention.

The robot is a 7 degrees of freedom (DOF) serial redundant manipulator composed of revolute joints. The reference frames were assigned following the standard DH convention, where each Z_i axis corresponds to the rotation axis of joint $i + 1$. The X_i axes were defined along the common normals between consecutive Z axes, while the origins were positioned at the intersection points between the joint axes and the common normals.

The resulting frame assignment is shown below, together with the orientation of all local coordinate systems and joint rotation directions.

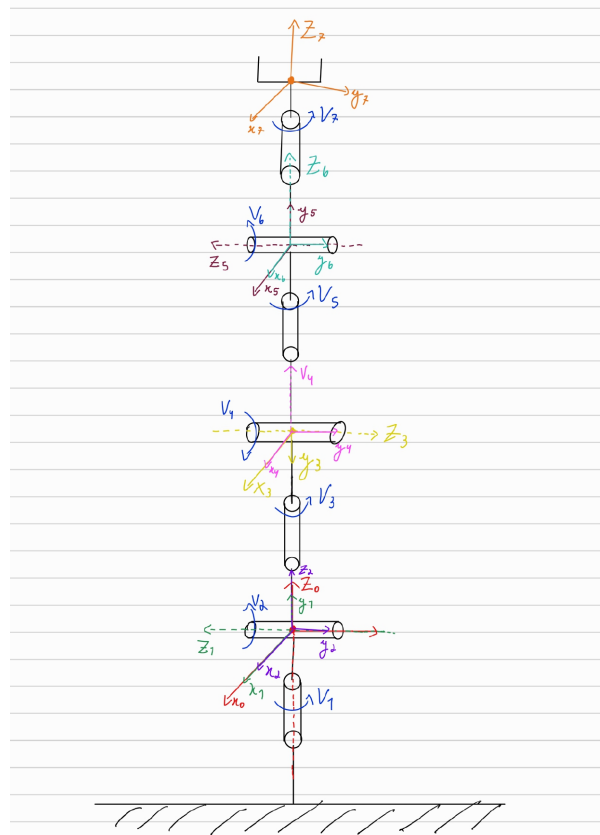


Figure 1: D-H Frame Assignment

The final Denavit-Hartenberg table obtained for the manipulator is presented below.

Joint i	d_i	θ_i	a_i	α_i	Offset
1	0	θ_1	0	$\pi/2$	0
2	0	θ_2	0	$-\pi/2$	0
3	d_3	θ_3	0	$-\pi/2$	0
4	0	θ_4	0	$\pi/2$	0
5	d_5	θ_5	0	$\pi/2$	0
6	0	θ_6	0	$-\pi/2$	0
7	d_7	θ_7	0	0	0

Table 1: Denavit-Hartenberg Parameters

There are two versions of the KUKA LBR MED, so as advised in the lab classes we choose the model **KUKA LBR iiwa 7 R800** to determine the d values. Based on that, the respective values of d used in our project were $d_3 = 0.400$ mm, $d_5 = 0.400$ mm, and $d_7 = 0.126$ mm.

2 Step 2: Direct Kinematics

In this step, we developed the Direct Kinematics (DK) model using the Denavit-Hartenberg (D-H) parameters obtained in Step 1.

The forward Transformation Matrix T_0^7 was computed symbolically by multiplying the individual transformation matrices A_i^{i+1} . Because the base reference frame (Frame 0) and the first joint frame (Frame 1) share the same origin in our convention, the parameter $d_1 = 0$.

2.1 Validation of Known Configurations

To validate the model, the analytical equations were tested against three known physical configurations. The resulting end-effector position p was compared to the geometric expectations. (*See Annex A for visual representations*).

Test Case	Joint Config q (rad)	Expected p (m)	Computed p (m)	Err (mm)
Home	[0, 0, 0, 0, 0, 0, 0]	[0; 0; 0.926]	[0; 0; 0.926]	0.00
Shoulder	[0, 1.57, 0, 0, 0, 0, 0]	[-0.926; 0; 0]	[-0.926; 0; 0]	0.00
L-Shape	[0, 0, 0, -1.57, 0, 0, 0]	[-0.526; 0; 0.4]	[-0.526; 0; 0.4]	0.00

Table 2: Validation of Direct Kinematics

2.2 Analysis

The Direct Kinematics model perfectly matched the geometric expectations with a numerical error of zero for all tested configurations. The result confirms that the D-H table and the subsequent matrix multiplications accurately represent the physical dimensions of the KUKA LBR MED.

3 Step 3: Inverse Kinematics

In this step, a closed-form analytical Inverse Kinematics (IK) solver was developed. Since the KUKA LBR MED is a 7-DOF redundant manipulator, the system is underconstrained for a standard 6-DOF task (position and orientation).

To obtain a deterministic closed-form solution, the redundancy was parameterized using the elbow angle, effectively decoupling the system. The wrist joints were then solved using an intrinsic Z-Y-Z Euler angle decomposition.

3.1 Validation of Known Configurations

The solver was validated using a "round-trip" test: a set of joint angles q_{orig} was used to generate a target pose T_{goal} via the DK model. This target was fed into the IK solver to produce a joint solution q_{inv} , which was then passed back through the DK model to obtain T_{check} . (See Annex A for visual representations).

Test Case	Original q_{orig} (rad)	IK Solution q_{inv} (rad)	Pos. Err (mm)	Ori. Err (rad)
Home	[0, 0, 0, 0, 0, 0, 0]	[0, 0, 0, 0, 0, 0, 0]	0.00	0.00
Elbow	[0, 0.78, 0, 1.04, 0, 0.52, 0]	[0, 0.78, 0, 1.04, 0, 0.52, 0]	$< 10^{-12}$	$< 10^{-14}$
Random	<i>Randomized bounded set</i>	<i>Solved valid set</i>	$< 10^{-12}$	$< 10^{-14}$

Table 3: Validation of Inverse Kinematics

3.2 Analysis

The analytical Inverse Kinematics successfully achieved the target poses with numerical precision (errors at the floating-point limit). In all tests, the orientation and position errors were strictly below 1 mm and 0.01 rad. Furthermore, the solver correctly enforced mathematical domain limits, preventing complex numbers when target points were outside the robot's physical workspace.

4 Step 4: Geometric Jacobian and Singularities

In this step, the geometric Jacobian was obtained from the direct kinematics model. Since all joints are revolute, each column of the geometric Jacobian was constructed from the joint rotation axis and the relative position between the joint origin and the end-effector. The upper part of each column represents the linear velocity contribution generated by the joint rotation, while the lower part represents the angular velocity contribution along the joint axis.

$$J_i = \begin{bmatrix} \hat{z}_{i-1} \times (p_e - p_{i-1}) \\ \hat{z}_{i-1} \end{bmatrix}$$

This gives us a 6×7 Jacobian matrix, which is expected because the robot has 7 joints but the end-effector motion is described by 6 spatial velocities.

4.1 Numerical Differentiation Validation

To check that the expression was correct, the analytical velocity was compared with a numerical derivative of the end-effector position along a smooth joint trajectory. The two

results were very close: the maximum linear velocity error was approximately 7.5×10^{-5} m/s, and the angular velocity error stayed below 1.2×10^{-3} rad/s.

4.2 Singularity Rank Analysis

The Jacobian was then evaluated in different robot configurations to study singularities.

Configuration	Main condition	Rank	Interpretation
Generic pose	Bent arm	6	Full task-space mobility
Shoulder singularity	$q_2 = 0$	6	Redundancy reduced, task motion preserved
Wrist singularity	$q_6 = 0$	6	Redundancy reduced, task motion preserved
Elbow singularity	$q_4 = 0$	5	One task-space direction is lost

Table 4: Singularity Rank Analysis

In Simulink, two blocks are used for this part of the project. The `Direct_Kinematics` block receives, separately, the seven joint angles q_1 to q_7 and computes the end-effector pose, returning the rotation matrix `Kuka_R` and position vector `Kuka_p`. The `Jacobian` block receives the joint vector q and returns the matrix `J_sym`, which maps joint velocities into end-effector linear and angular velocity. Together, these blocks connect the position model from the previous step with the velocity model required for singularity analysis and later control.

The most important case was the elbow singularity, shown in the figure below. When the arm is stretched and $q_4 = 0$, the jacobian rank drops from 6 to 5. Physically, this means the robot can no longer generate motion in one direction at the end-effector, especially along the stretched arm direction, being in a state of singularity.

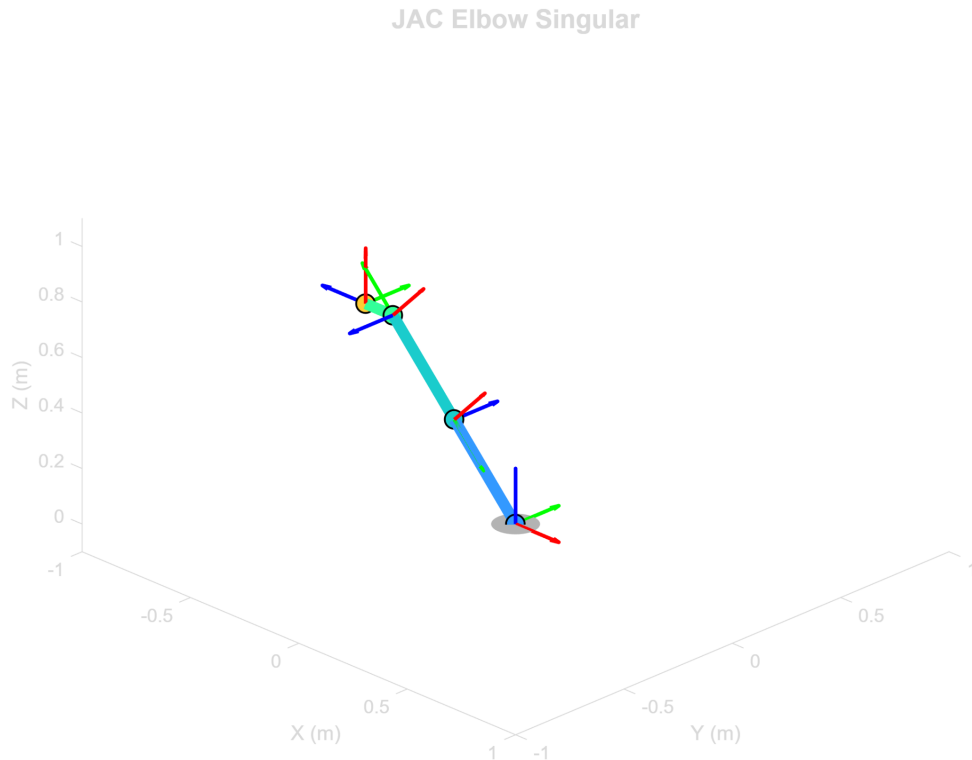


Figure 2: Elbow Singularity Configuration

Overall, our results show that the Jacobian model behaves as expected, correctly mapping the joint velocities into end-effector velocities, and its rank analysis identifies the elbow-stretched pose as the critical singularity for the robot arm.

5 Step 5: Closed-Loop Inverse Kinematics (CLIK)

In this step, we implemented the Closed-Loop Inverse Kinematics controller using the forward kinematics and geometric Jacobian models developed in the previous steps. The main point behind CLIK is that instead of computing one closed-form joint solution directly, the robot continuously corrects its joint velocity from the measured task-space error.

The controller was implemented in `clik_controller.m` and integrated into the Simulink model `Kuka_CLIK_Model.slx`. At each simulation instant, the current end-effector pose is computed using forward kinematics.

Since this model of KUKA robot is redundant, the controller uses the damped pseudoinverse of the Jacobian together with null-space stabilization:

$$\dot{q} = J^\# \dot{x}_{ref} + (I - J^\# J) \dot{q}_0, \quad \dot{q}_0 = -K_0(q - q_{rest})$$

In the code, the null-space gain is named `Kn`, but in this report it is written as K_0 . The baseline value was $K_0 = 0.15I_7$, with $q_{rest} = 0$, so that the robot is gently pulled toward a central resting posture without changing the final end-effector pose.

The final Simulink model for the Closed-Loop Inverse Kinematics is below:

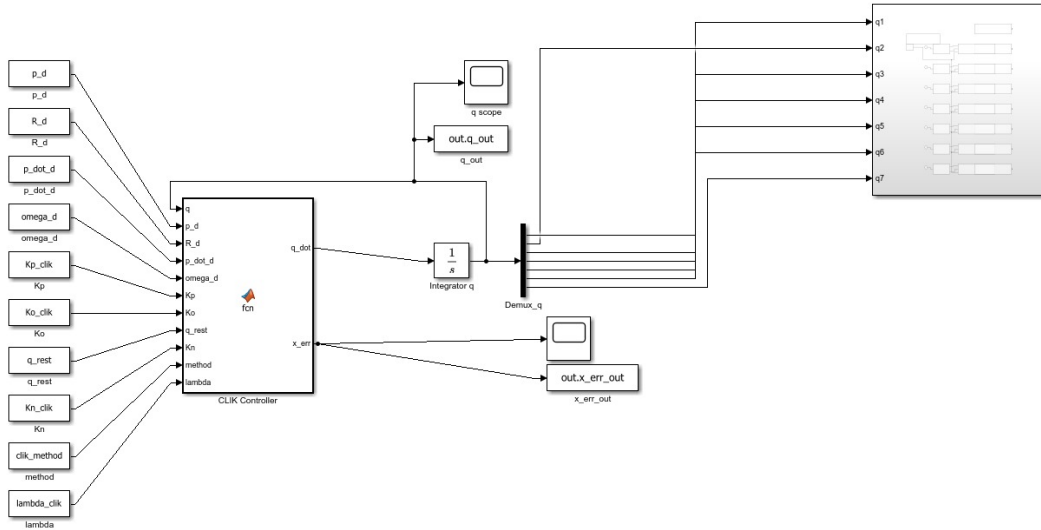


Figure 3: Simulink Block Diagram

In the model, the desired position, orientation, and task-space velocities are provided as inputs to the CLIK controller block. Using the current joint configuration, the controller computes the forward kinematics and the geometric Jacobian of the manipulator, evaluates the errors, and generates the joint velocity vector $\dot{q}$. The joint velocities are integrated to obtain the updated joint positions q , which are then fed back into the controller, closing the feedback loop.

In the graphic below, we can observe the convergence of the position and orientation, as the error decreases to almost 0, after 1 second.

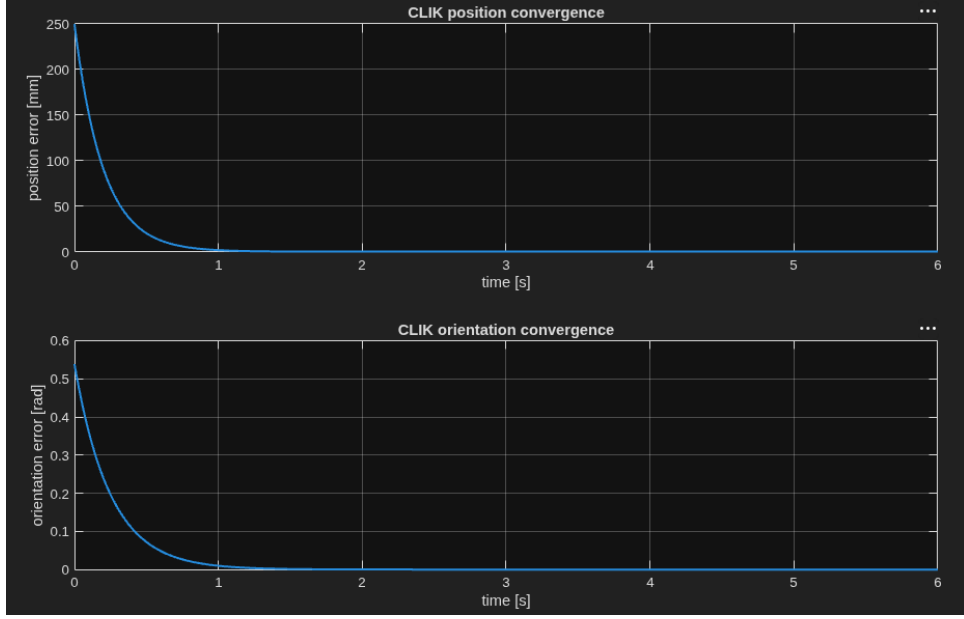


Figure 4: Convergence Graphic

5.1 Validation Against Analytical Inverse Kinematics

First, a target pose (p_d, R_d) was generated from a known joint configuration through forward kinematics, and then tested with the analytical inverse kinematics model from Step 3.

Method	Final Position Error	Final Orientation Error
Analytical IK + DK	1.18×10^{-13} mm	3.53×10^{-16} rad
CLIK, baseline case	1.58×10^{-4} mm	2.06×10^{-8} rad

Table 5: CLIK vs Analytical IK

The CLIK result does not reach the same numerical zero as the analytical solution because it is a dynamic method, integrated over time, the robot is redundant, and the Jacobian inversion is damped. However, the final error is far below 1 mm.

$$q_{goal} = [0.25 \quad 0.70 \quad -0.35 \quad 1.05 \quad 0.25 \quad -0.55 \quad 0.15]^T \text{ rad}$$

5.2 Influence of the Proportional Gain

Three proportional gain values were tested while keeping the same target, the same initial configuration, and $K_0 = 0.15$. The purpose was to observe how the gain affects convergence speed, stability, and joint effort.

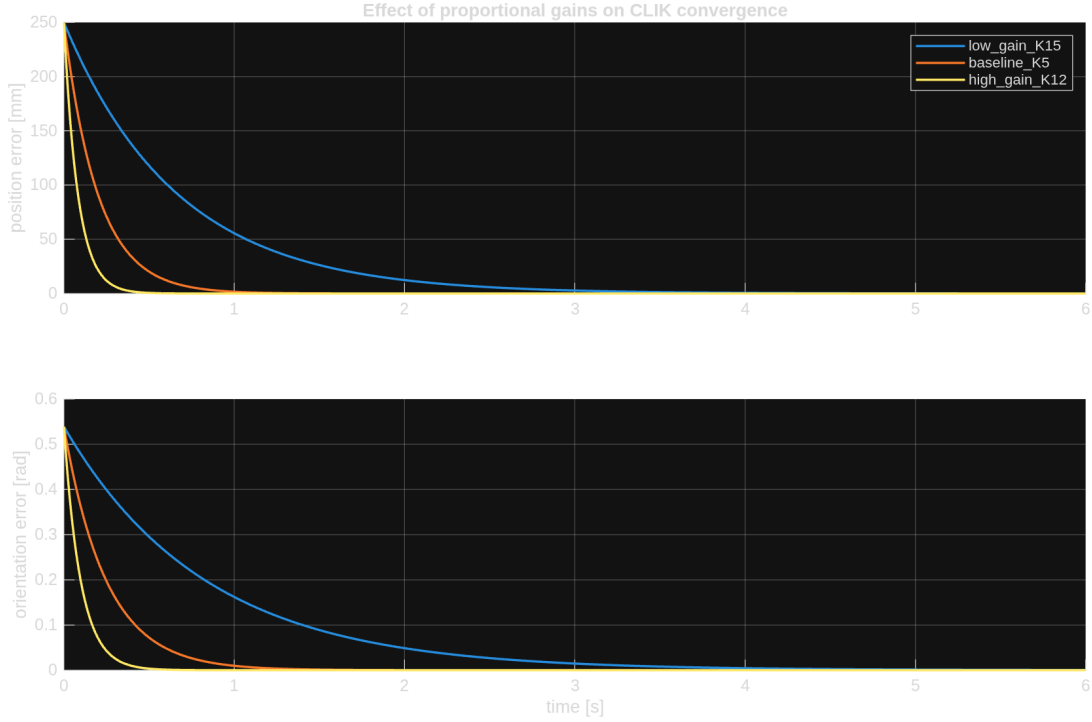


Figure 5: Influence of Proportional Gain

The low-gain case was stable, but it took longer to converge. The high-gain case converged much faster, but it required larger joint velocities. For the final model, the baseline case is a good compromise: it converges in approximately 1.1 s without producing joint velocities as aggressive as the high-gain case.

5.3 Influence of the Initial Joint Configuration

The redundancy of the manipulator was tested again using the same target pose, but with a different initial joint configuration:

$$q_0 = [-0.60 \ 0.95 \ 0.70 \ 0.85 \ -0.55 \ 0.65 \ -0.35]^T \text{ rad}$$

Test Case	Pos. Err (mm)	Ori. Err (rad)	Set. Time (s)	Max Joint (rad)
Baseline q_0	0.000158	1.0×10^{-8}	1.100	1.053
Different q_0	0.000157	1.0×10^{-8}	1.356	1.457

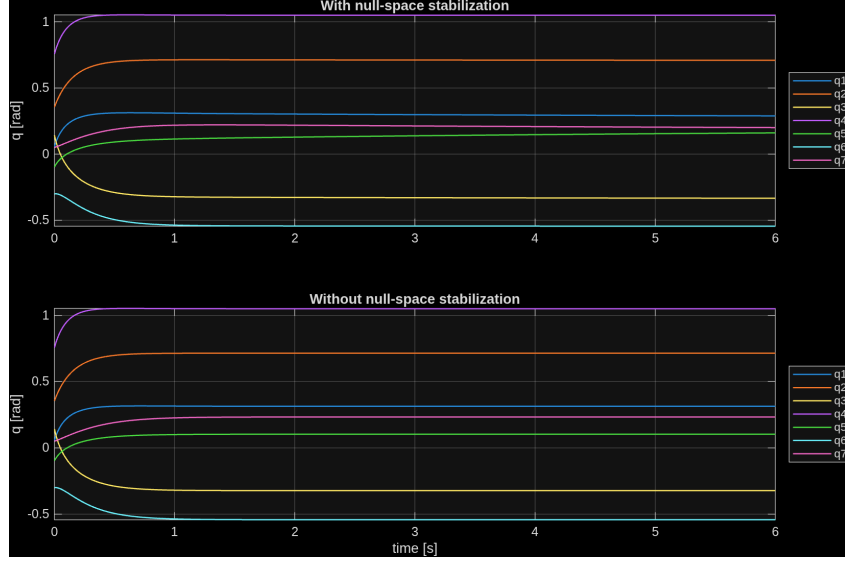
Table 6: Influence of Initial Configuration

Both tests converged to the same Cartesian pose, but with different joint trajectories. This result confirms the expected behaviour of a 7-DOF robot: the end-effector task is satisfied, while the internal posture depends on the initial configuration and on the null-space term.

5.4 Null-Space Gain K_0

The next test compares the controller with and without null-space stabilization. In both cases, the end-effector converges to the desired target, but the internal joint posture

changes.



Test Case	K_0 Gain	Pos. Err (mm)	Set. Time (s)	Final dist. to q_{rest}
With null-space	0.15	0.000158	1.100	1.4705
Without null-space	0.00	0.000000	1.100	1.4747

Table 7: Influence of Null-Space Gain

For this specific target, the final numerical difference is small because the motion is short and the initial posture is not very far from the resting posture. Even so, the joint plot shows that the null-space term changes how the redundancy is used. During repeated CLIK updates, this term prevents redundant joints from drifting toward uncomfortable configurations or mechanical limits.

5.5 Final Discussion

Our tests lead to three main conclusions:

1. **Validation:** CLIK validates both the forward kinematics and the Jacobian because it converges to the same pose as the analytical inverse kinematics with a very small error.
2. **Proportional Gain:** The proportional gain directly controls the trade-off between speed and joint effort: larger gains reduce settling time, but increase joint velocities ($\dot{q}$).
3. **Null-Space Stabilization:** The redundancy of the robot makes the null-space term important for controlling the internal posture without disturbing the main end-effector task.

Annex A: Visual Representation of Test Cases

Test Cases for Direct Kinematics:

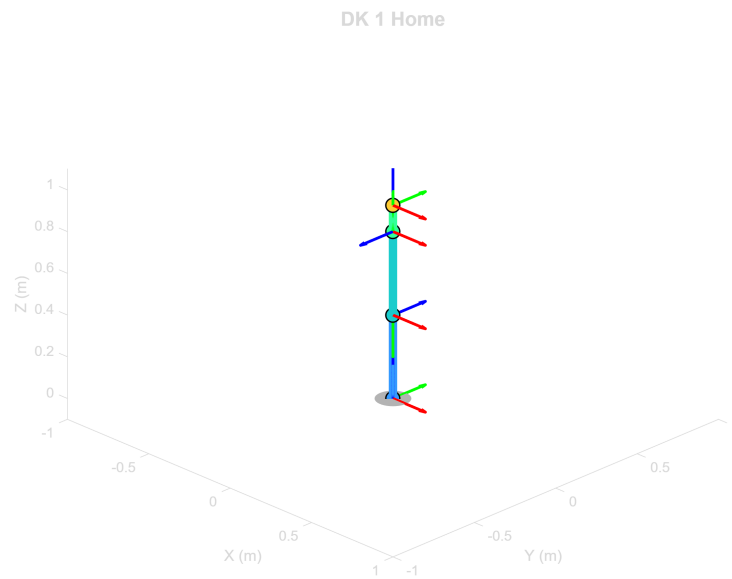


Figure 6: Figure A.1: Home (Fully stretched)

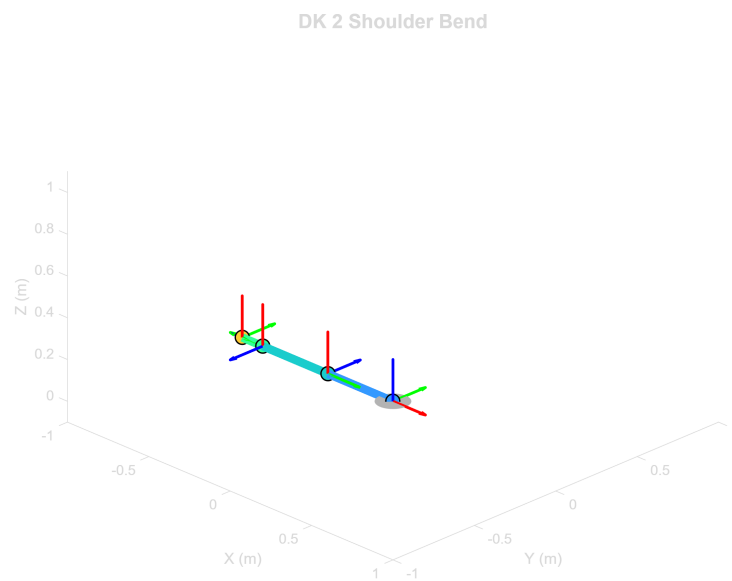


Figure 7: Figure A.2: Shoulder Bend

DK 3 LShape Bend

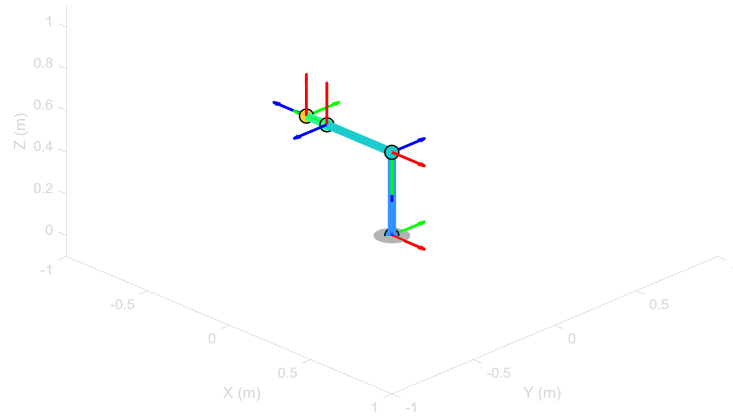


Figure 8: Figure A.3: L-Shape Bend

Test Cases for Inverse Kinematics:

IK 2 Elbow Bend

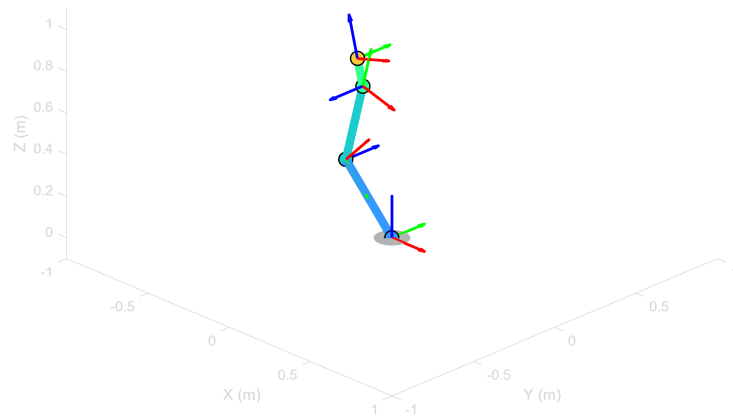


Figure 9: Figure A.4: Inverse Kinematics Elbow Bend